WILDTRAIL

SAFARI BOOKING SYSTEM

SE1020 | Object Oriented Programming

Group 26-MTR-Y1S2-WD-009

Project design, implementation, verification and Git contribution record

Project overview

WildTrail is a web application for browsing Sri Lankan safari packages and managing reservations. A Java 17 Spring Boot API implements the business rules and data operations. A Next.js website provides customer pages and a staff console. The project is a campus demonstration: catalogue records are samples, card payment does not collect card details or charge money, and example reviews are labeled as fictional.

Objectives and implemented functions

Component	Customer or staff functions	Persistence
Users and customers	Register, sign in/out, edit profile, manage staff accounts and customer records	SQL
Safari tours	Browse destinations and packages; create, edit, and manage package records	SQL
Bookings	Check availability; create, view, change, and cancel eligible bookings	SQL
Guides and vehicles	Manage vehicles, guides, assignments, and schedules	SQL for vehicles and assignments; **resources.txt** for guides
Payments	Track pending cash/card demonstration, record collection or refund, view receipts	SQL
Reviews	Submit, moderate, edit, and delete reviews; manage contact inquiries	SQL

The booking API calculates prices and validates dates, guest limits, availability, vehicle conflicts, ownership, and the 24-hour change/cancellation rule. Customers cannot directly change another customer's booking. Payment and booking updates use transactions. Vehicle and schedule locks protect concurrent changes. Historical records are restricted from deletion where removing them would break related bookings or payments.

Design and object-oriented concepts

Controllers accept requests, services enforce rules, domain classes hold validated state, and repositories handle SQL or text files. The class diagram and technical overview show the principal relationships.

- Encapsulation and information hiding: **Booking**, **User**, **Guide**, **Review**, **SafariPackage**, and **Vehicle** validate data before it is stored or changed.
- Abstraction: abstract **SafariPackage**, **Vehicle**, and **Payment** classes expose shared behavior. **PaymentProcessor** defines the processing contract.
- Inheritance: customer/admin users, safari types, jeep types, naturalist guides, and verified reviews extend their parent models.
- Polymorphism: package subtypes calculate prices through overridden behavior; vehicle and payment subtypes provide their own behavior behind shared interfaces.
- Composition: **BookingService** coordinates package, vehicle, booking, customer, and payment collaborators instead of placing every rule in a controller.

File handling and database

The local default is a persistent H2 SQL database; **database/schema.sql** and **database/data.sql** define the schema and sample catalogue. This uses the assignment's permitted SQL storage option. The guide module separately demonstrates real file CRUD: **FileGuideRepository** reads and writes UTF-8 JSON in **resources.txt**, replaces the file atomically, and preserves the original when reading fails. A clearly fictional guide sample file is supplied for demonstrations. The staff report feature writes text snapshots; these snapshots are secondary exports, not the authoritative database.

Each local installation creates its own administrator credentials through **scripts/setup.ps1**. Credentials, local databases, build output, and logs are excluded from Git. The source repository contains no required shared demo password.

Interface and main demonstration

1. Browse destinations and safari packages, then register or sign in as a customer.
2. Choose a package, date, guests, and vehicle; confirm a booking and view its reference.
3. Change or cancel an eligible booking and observe the server's availability and 24-hour checks.
4. Sign in as staff to manage packages, vehicles, guides, bookings, payments, reviews, and contact messages.
5. Create, read, update, and delete a sample guide, then inspect **resources.txt** and reload the guide list.
6. Record a cash payment, view the receipt, and moderate a customer review.

The viva guide identifies the relevant Java classes, business rules, and demonstrations for each module.

Team responsibilities and Git evidence

The table records module ownership and a specific contribution visible in the repository history. It does not attribute every line of the integrated baseline to the named member. GitHub commit links provide the verifiable individual evidence; merge commits and shared integration work are separate.

Member	Name	Module	Recorded change and review trail
IT26100894	Mekhala B.G.R.S	Users and customers	Count paid payments in customer total spent, PR #14
IT26100895	Mihiran S.P.D	Safaris and packages	Prevent packages extending past midnight, PR #12
IT26100896	Nayanajith W.R.S.C	Bookings and reservations; integration	Enforce 24-hour rescheduling notice, validate availability and cancellation, PR #7, PR #9
IT26100899	Senevirathna M.K.K.T	Guides and vehicles	Prevent incompatible vehicle updates, PR #16
IT26100900	Lakshani W.G.C	Payments and receipts	Recheck failed payment status before deletion, PR #10
IT26100901	Gamage C.I.P	Reviews and feedback	Validate review country length, PR #11

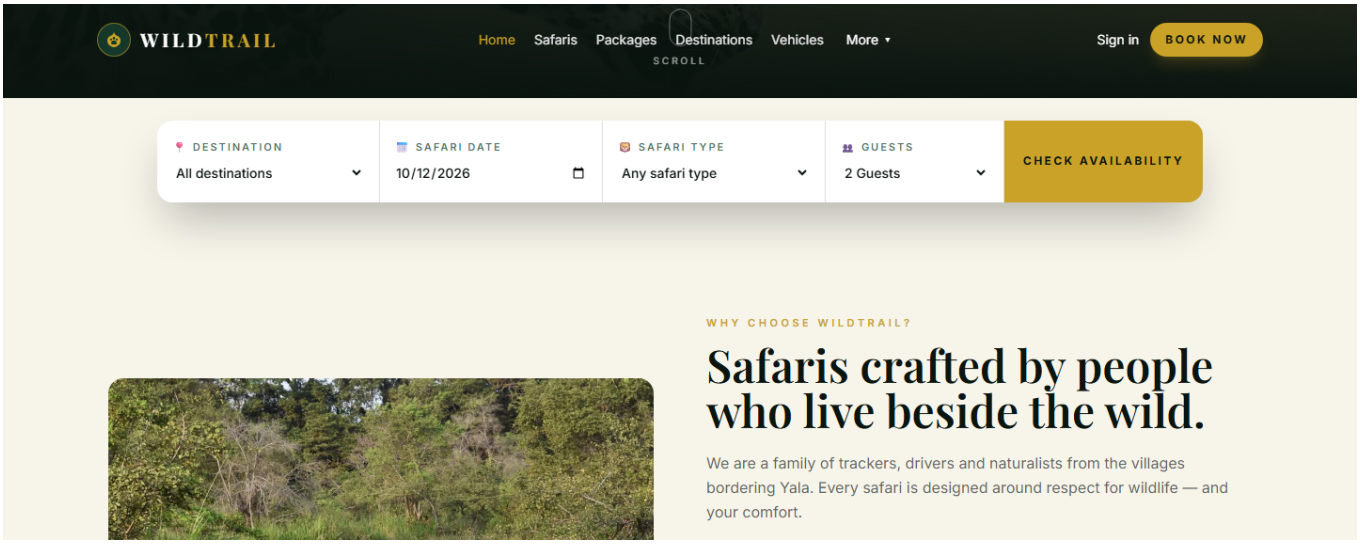
The full GitHub commit history and merged pull requests show the collaboration record. Detailed component ownership and source links are in the workload distribution and member guides under **team/**.

Verification and results

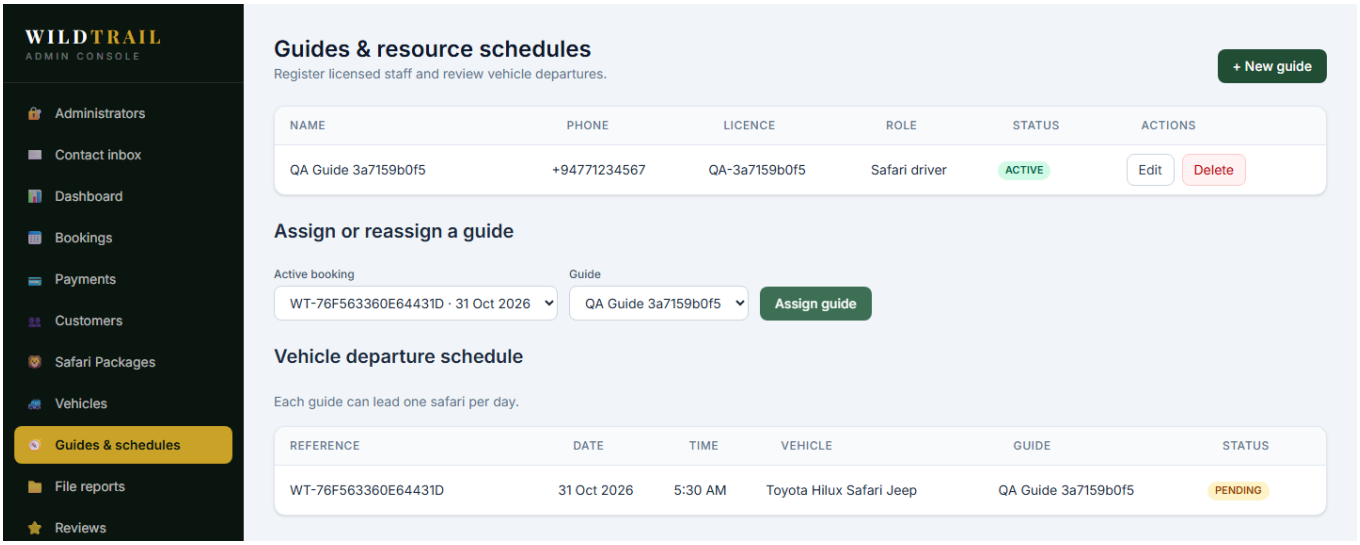
On 11 October 2026, the Java 17 Maven suite passed 57 tests. Frontend TypeScript, ESLint, and the production build passed. Browser QA against a disposable local H2 database passed 11 customer/staff flow checks; a separate navigation/authentication suite passed 4 checks, and responsive checks covered 320–2133 px widths. Tracked Markdown links were checked: 172 links across 17 files, with no missing local targets. The QA report records commands, scope, and limits.

These results establish local behavior under the tested configuration. They do not establish live payment processing, email delivery, live park availability, or production hosting. Each member must demonstrate and explain their own Java module during the viva.

Selected interface evidence



Customer discovery and booking search at desktop width.



Staff guide assignment using disposable demonstration records.